DESIGN AND ANALYSIS OF ALGORITHMS

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE CODE & NAME:

CSA06 – DESIGN AND ANALYSIS OF ALGORITHMS

ASSESSMENT CO4B-AT1–: ALGORITHM ENGINEERING –

IMPLEMENT, TEST, DEBUG & OPTIMISE

Submitted by: S.Harshavardhan

Registration Number:192524106

B.Tech – Artificial Intelligence and Data Science

SIMATS Engineering

Introduction, Environment and Engineering Approach

This report answers all six tasks of Assignment 4 Part II. Every result quoted here was produced by running the programs listed in the report; nothing was typed in by hand. All programs are written in Python 3.12 and run from the command line on a standard laptop-class machine. Each task follows the workflow demanded by the assignment: **DESIGN → IMPLEMENT → INSTRUMENT → TEST → BREAK → DEBUG → VALIDATE → DEFEND**. For every task the report gives the source code, the test input, the expected behaviour, the actual output, intermediate algorithmic evidence (DP tables, traces, counters), a boundary or failure test, and an interpretation, which is the evidence chain in the mandatory submission list.

Software engineering conventions. Each program separates algorithm logic from input and output: functions such as word_wrap, knapsack_dp, bellman_ford, schedule, kruskal and solve_tsp only compute and return data structures (tables, parent pointers, counters), while separate functions such as report, show_case, print_result and show_matrix print them. Names are descriptive, assumptions are stated at the top of each section, invalid input raises ValueError instead of returning a misleading answer, and every random test uses a fixed seed so that all results are reproducible. Instrumentation (counters and traces) is built into the algorithms rather than added afterwards, so the evidence displayed is the evidence actually used by the algorithm.

Report map. Task 1 (word wrap DP, 8 marks); Task 2 (greedy versus DP knapsack, 9); Task 3 (Dijkstra, Bellman-Ford, Floyd, Warshall, 10); Task 4 (assembly-line DP, 7); Task 5 (Kruskal, Prim, Borůvka, 7); Task 6 (TSP by DP, 9).

Task 1 – Word Wrap as a Document-Layout Engine (8 Marks)

1.1 Implementation of the Dynamic Programming Solution

Model. The text is a list of n words and every line holds at most W characters, with one space between words. A non-final line with s unused spaces costs s^2. The last line is free. The objective is the arrangement with minimum total penalty. **Assumptions:** words are separated by exactly one space, no word may be longer than W (the program rejects such input), and a line may not be empty.

DP formulation. Let best[i] be the minimum penalty for laying out the suffix words[i..n-1] when line breaking starts at word i. With best[n] = 0 the recurrence is: best[i] = min over all j such that words i..j fit on one line of cost(i, j), where cost(i, j) = 0 if j = n−1 (final line, no penalty) and cost(i, j) = $(W - \text{length}(i..j))^2$ + best[j+1] otherwise. A parallel array choice[i] stores the winning j+1, which is the index of the first word of the next line. Working from the end of the text towards the start makes the free-final-line rule natural, because the line that contains the last word is always evaluated with cost 0. Optimal substructure holds because once the first line is fixed, the remaining words form an independent smaller problem whose best solution is already stored in best[j+1]. The running time is O(n·k), where k is the maximum number of words that fit on one line, and the space is O(n).

Listing 1 – task1_wordwrap.py (source code)

```
"""Task 1 - Word wrap by dynamic programming (minimise sum of squared trailing spaces)."""
```

```
def validate_input(words, width):
    if width <= 0:
        raise ValueError("line width must be positive")
    for word in words:
        if len(word) > width:
            raise ValueError(f"word '{word}' (length {len(word)}) exceeds width {width}")


def line_length(words, first, last):
    """Characters needed for words[first..last] separated by single spaces."""
    return sum(len(w) for w in words[first:last + 1]) + (last - first)


def word_wrap(words, width):
    """best[i] = minimum penalty for arranging words[i:]; choice[i] = first word of the next line."""
    validate_input(words, width)
    n = len(words)
    best = [float("inf")] * n + [0]
    choice = [n] * (n + 1)
    candidates = {}                    # instrumentation: every option evaluated for state i
    for i in range(n - 1, -1, -1):
        candidates[i] = []
        last = i
        while last < n and line_length(words, i, last) <= width:
            spaces = width - line_length(words, i, last)
            penalty = 0 if last == n - 1 else spaces ** 2 + best[last + 1]
            candidates[i].append((last, spaces, penalty))
            if penalty < best[i]:
                best[i], choice[i] = penalty, last + 1
            last += 1
    return best, choice, candidates


def build_lines(words, width, choice):
    lines, i, n = [], 0, len(words)
    while i < n:
        j = choice[i]
        text = " ".join(words[i:j])
        spaces = width - len(text)
        penalty = 0 if j == n else spaces ** 2
        lines.append((text, spaces, penalty))
        i = j
```

```
    return lines


def greedy_wrap_penalty(words, width):
    """Reference: classic first-fit wrapping, used only for comparison."""
    lines, current = [], words[0] if words else ""
    for w in words[1:]:
        if len(current) + 1 + len(w) <= width:
            current += " " + w
        else:
            lines.append(current)
            current = w
    lines.append(current)
    return sum((width - len(l)) ** 2 for l in lines[:-1])


def report(words, width, show_states=False):
    best, choice, candidates = word_wrap(words, width)
    lines = build_lines(words, width, choice)
    print(f"--- width W = {width} ---")
    if show_states:
        print("DP states (i, word, best[i], next line starts at, options (last word, spaces, total))")
        for i in range(len(words)):
            opts = ", ".join(f"({l},{s},{p})" for l, s, p in candidates[i])
            print(f"i={i:2d} {words[i]:<12s} best={best[i]:>4} next={choice[i]:2d}  {opts}")
    print(f"Minimum total penalty = {best[0]}")
    for k, (text, spaces, penalty) in enumerate(lines, 1):
        print(f"Line {k}: |{text:<{width}}| unused={spaces:2d} penalty={penalty}")
    return best[0], [l[0] for l in lines]


if __name__ == "__main__":
    text = "Dynamic programming turns a large layout problem into small decisions that reuse earlier answers"
    words = text.split()
    print("Words:", len(words), words)
    report(words, 24, show_states=True)
    for w in (20, 28, 36):
        report(words, w)
    for w in (24, 33):
        print(f"\nW={w}: greedy first-fit penalty = {greedy_wrap_penalty(words, w)}, DP penalty = {word_wrap(words, w)[0][0]}")
    report(words, 33)
    print("\nSweep of widths (W: DP penalty / greedy penalty):")
```

```
    print("  " + "  ".join(f"{w}: {word_wrap(words, w)[0][0]}/{greedy_wrap_penalty(words,
w)}" for w in range(30, 37)))
    print("\n=== Boundary tests ===")
    report(["abcd", "efghij"], 6)                    # word exactly equals width
    report(["alpha", "beta", "gamma"], 20)           # everything fits on one line
    report(["solo"], 4)                              # single word, width == length
    best, _, _ = word_wrap([], 10)
    print("Empty text penalty:", best[0])
    try:
        word_wrap(["extraordinary"], 5)
    except ValueError as err:
        print("Rejected:", err)
```

Test input. The text “Dynamic programming turns a large layout problem into small decisions that reuse earlier answers” (14 words) with a baseline width W = 24. **Expected behaviour:** a five-line arrangement whose penalties add up to the minimum; the final line must show penalty 0 even though it has unused spaces.

Output 1 – baseline run, W = 24 (DP states, arrangement and penalties)

```
Words: 14 ['Dynamic', 'programming', 'turns', 'a', 'large', 'layout', 'problem', 'into', 'small',
'decisions', 'that', 'reuse', 'earlier', 'answers']
--- width W = 24 ---
DP states (i, word, best[i], next line starts at, options (last word, spaces, total))
i= 0 Dynamic      best=  93 next= 2  (0,17,382), (1,5,93)
i= 1 programming  best=  93 next= 4  (1,13,237), (2,7,105), (3,5,93)
i= 2 turns        best=  68 next= 6  (2,19,417), (3,17,357), (4,11,162), (5,4,68)
i= 3 a            best=  56 next= 7  (3,23,597), (4,17,330), (5,10,152), (6,2,56)
i= 4 large        best=  68 next= 7  (4,19,402), (5,12,196), (6,4,68)
i= 5 layout       best=  41 next= 8  (5,18,376), (6,10,152), (7,5,41)
i= 6 problem      best=  52 next= 9  (6,17,341), (7,12,160), (8,6,52)
i= 7 into         best=  52 next=10  (7,20,416), (8,14,212), (9,4,52)
i= 8 small        best=  16 next=11  (8,19,377), (9,9,117), (10,4,16)
i= 9 decisions    best=  16 next=12  (9,15,261), (10,10,100), (11,4,16)
i=10 that         best=  36 next=13  (10,20,400), (11,14,196), (12,6,36)
i=11 reuse        best=   0 next=14  (11,19,361), (12,11,121), (13,3,0)
i=12 earlier      best=   0 next=14  (12,17,289), (13,9,0)
i=13 answers      best=   0 next=14  (13,17,0)
Minimum total penalty = 93
Line 1: |Dynamic programming     | unused= 5 penalty=25
Line 2: |turns a large layout    | unused= 4 penalty=16
Line 3: |problem into small      | unused= 6 penalty=36
Line 4: |decisions that reuse    | unused= 4 penalty=16
Line 5: |earlier answers         | unused= 9 penalty=0
```

Result. The minimum total penalty is **93**. The arrangement and the per-line penalties are 25 + 16 + 36 + 16 + 0 = 93. The program therefore reports not only the cost but the words on each line, the unused spaces on each line and the penalty contributed by each line, as the task requires.

1.2 Exposing the DP Behaviour

The program prints one row for every DP state i. Each row contains the word at which the suffix starts, the value best[i], the value next (= choice[i]), and the list of options evaluated for that state, written as (last word index j, unused spaces s, total cost). The total cost of an option is s^2 + best[j+1], or 0 when j is the last word. These rows show how the optimum was built from the bottom up.

- **best[i]** means the least possible penalty for the part of the document that begins at word i; best[13] = best[12] = best[11] = 0 because “answers”, “earlier answers” and “reuse earlier answers” each fit on the final line, which is free.
- **next** is the index where the following line begins; following next from i = 0 gives 0 → 2 → 6 → 9 → 12 → 14, which are the five lines of the answer.
- **An option (j, s, total)** is one candidate first line. At i = 8 (“small”) the options are (8, 19, 377), (9, 9, 117) and (10, 4, 16). The first puts only “small” on the line (19^2 = 361, plus best[9] = 16 gives 377); the second adds “decisions” (9^2 = 81 plus best[10] = 36 gives 117); the third adds “that” (4^2 = 16 plus best[11] = 0 gives 16). The minimum, 16, is stored in best[8] and next = 11.
- **Reuse of earlier answers** is visible at i = 2: the winning option (5, 4, 68) is 4^2 = 16 plus best[6] = 52. The value 52 was computed earlier, at i = 6, and is simply looked up. Likewise best[0] = 93 = 5^2 + best[2] = 25 + 68. The final answer is assembled entirely from smaller stored answers, which is the optimal-substructure property made visible.

1.3 Requirement Change – Three Different Line Widths

The same text was run with W = 20, 28 and 36. The predictions below were written before each run, using the baseline arrangement (line lengths 19, 20, 18, 20 and 15 characters) and the baseline penalty of 93.

Width	Prediction made before execution	Observed arrangement	Observed penalty	Prediction correct?
20	Arrangement stays the same because every baseline line already fits in 20 characters; no change point; penalty falls because the trailing slack shrinks.	Identical five lines	5 (1+0+4+0+0)	Yes, on all three points
28	Arrangement changes; first change on line 1, because “turns” (25 characters in total) now fits; penalty expected to fall.	4 lines; line 1 = “Dynamic programming turns”	14	Yes

36	Arrangement changes on line 1 (“a large” also fits); fewer lines (about 3); penalty expected to fall.	3 lines; line 1 = “Dynamic programming turns a large”	10	Yes

Output 2 – runs for W = 20, 28 and 36, the greedy comparison at W = 33, the width sweep and the boundary tests

```
--- width W = 20 ---
Minimum total penalty = 5
Line 1: |Dynamic programming | unused= 1 penalty=1
Line 2: |turns a large layout| unused= 0 penalty=0
Line 3: |problem into small  | unused= 2 penalty=4
Line 4: |decisions that reuse| unused= 0 penalty=0
Line 5: |earlier answers     | unused= 5 penalty=0
--- width W = 28 ---
Minimum total penalty = 14
Line 1: |Dynamic programming turns   | unused= 3 penalty=9
Line 2: |a large layout problem into | unused= 1 penalty=1
Line 3: |small decisions that reuse  | unused= 2 penalty=4
Line 4: |earlier answers             | unused=13 penalty=0
--- width W = 36 ---
Minimum total penalty = 10
Line 1: |Dynamic programming turns a large   | unused= 3 penalty=9
Line 2: |layout problem into small decisions | unused= 1 penalty=1
Line 3: |that reuse earlier answers          | unused=10 penalty=0

W=24: greedy first-fit penalty = 93, DP penalty = 93

W=33: greedy first-fit penalty = 89, DP penalty = 65
--- width W = 33 ---
Minimum total penalty = 65
Line 1: |Dynamic programming turns a      | unused= 6 penalty=36
Line 2: |large layout problem into small  | unused= 2 penalty=4
Line 3: |decisions that reuse earlier     | unused= 5 penalty=25
Line 4: |answers                          | unused=26 penalty=0

Sweep of widths (W: DP penalty / greedy penalty):
  30: 50/50  31: 25/25  32: 42/42  33: 65/89  34: 94/118  35: 4/4  36: 10/10

=== Boundary tests ===
--- width W = 6 ---
Minimum total penalty = 4
Line 1: |abcd  | unused= 2 penalty=4
Line 2: |efghij| unused= 0 penalty=0
```

```
--- width W = 20 ---
Minimum total penalty = 0
Line 1: |alpha beta gamma    | unused= 4 penalty=0
--- width W = 4 ---
Minimum total penalty = 0
Line 1: |solo| unused= 0 penalty=0
```

Interpretation. All three predictions were confirmed, but the experiment also showed something a prediction based only on intuition would miss: the penalty is **not monotonic in the width**. W = 20 gives 5 while W = 28 gives 14, and the sweep shows W = 33 → 65, W = 34 → 94, but W = 35 → 4. A wider line does not guarantee a better layout, because the penalty depends on how well the natural word lengths tile the width. The sweep also shows the value of DP over a simple first-fit (greedy) wrapper: at W = 33 first-fit gives 89 while DP gives 65, and at W = 34 it gives 118 against 94. At W = 24 both agree at 93. First-fit commits to filling each line as much as possible without regard to what the next line will look like, which is exactly the decision the DP postpones until all alternatives have been priced.

1.4 Boundary Test

Selected case: a word whose length is exactly equal to the line width (words “abcd” and “efghij”, W = 6). I selected it because the most likely implementation defect in this program is an off-by-one error in the fit test: writing “length < W” instead of “length <= W” would make “efghij” appear not to fit anywhere, so the program would either crash, loop forever or return infinity. **Expected:** “efghij” sits alone on a full line (unused 0, penalty 0) and “abcd” on its own line before it (unused 2, penalty 4), so the total is 4. **Actual:** exactly this (see the boundary section of Output 2). Four further boundary tests were run: all words fitting on a single line (penalty 0, because the final line is free), a single word with W equal to its length (0), empty text (0 penalty, no lines), and a word longer than W (“extraordinary” with W = 5), which is rejected with a ValueError rather than silently producing an infinite penalty. These tests give confidence that the free final line, the equality case and the invalid-input case are all handled correctly.

Task 2 – Greedy versus Dynamic Programming: Build It, Break It, Explain It (9 Marks)

2.1 Implementation and Normal Case

Solution A is the fractional knapsack: items are sorted by value-to-weight ratio and taken in that order, splitting the last item if necessary. **Solution B** is the 0/1 knapsack by dynamic programming: table[i][c] is the best value using the first i items with capacity c, with table[i][c] = max(table[i−1][c], table[i−1][c−w_i] + v_i). Both report the actual items chosen (with fractions for A), and the DP recovers its selection by walking back through the table. A third helper, greedy_01, applies the same ratio rule to the 0/1 problem (take an item whole if it fits, otherwise skip it); it is the object of the counterexample in 2.2.

Listing 2 – task2_knapsack.py (source code)

```
"""Task 2 - Fractional knapsack (greedy) vs 0/1 knapsack (dynamic programming)."""
import random
from functools import cmp_to_key
```

```
def ratio_order(items):
    """Sort by value/weight, highest first; returns (ordered items, comparison count)."""
    counter = {"comparisons": 0}

    def compare(a, b):
        counter["comparisons"] += 1
        ra, rb = a[2] / a[1], b[2] / b[1]
        return (rb > ra) - (rb < ra)

    return sorted(items, key=cmp_to_key(compare)), counter["comparisons"]


def fractional_greedy(items, capacity):
    ordered, comparisons = ratio_order(items)
    remaining, total, chosen, steps = capacity, 0.0, [], []
    for item_id, weight, value in ordered:
        if remaining <= 0:
            break
        fraction = min(1.0, remaining / weight)
        gained = value * fraction
        remaining -= weight * fraction
        total += gained
        chosen.append((item_id, fraction))
        steps.append(f"take {fraction:.2f} of {item_id} (ratio {value / weight:.2f}) -> value
+{gained:.1f}, capacity left {remaining:.1f}")
    return total, chosen, steps, {"comparisons": comparisons, "decisions": len(steps),
"memory_cells": len(items)}


def greedy_01(items, capacity):
    """Same ratio intuition, but an item is taken whole or not at all."""
    ordered, comparisons = ratio_order(items)
    remaining, total, chosen, steps = capacity, 0, [], []
    for item_id, weight, value in ordered:
        if weight <= remaining:
            remaining -= weight
            total += value
            chosen.append(item_id)
            steps.append(f"take {item_id} (w={weight}, v={value}, ratio {value / weight:.2f}),
capacity left {remaining}")
        else:
            steps.append(f"skip {item_id} (w={weight} > capacity left {remaining})")
```

```
    return total, chosen, steps, {"comparisons": comparisons, "decisions": len(ordered),
"memory_cells": len(items)}


def knapsack_dp(items, capacity):
    n = len(items)
    table = [[0] * (capacity + 1) for _ in range(n + 1)]
    cell_updates = 0
    for i, (_, weight, value) in enumerate(items, 1):
        for cap in range(capacity + 1):
            cell_updates += 1
            table[i][cap] = table[i - 1][cap]
            if weight <= cap and table[i - 1][cap - weight] + value > table[i][cap]:
                table[i][cap] = table[i - 1][cap - weight] + value
    chosen, cap = [], capacity
    for i in range(n, 0, -1):
        if table[i][cap] != table[i - 1][cap]:
            chosen.append(items[i - 1][0])
            cap -= items[i - 1][1]
    return table[n][capacity], chosen[::-1], {"cell_updates": cell_updates, "memory_cells": (n + 1)
* (capacity + 1)}


def show_case(title, items, capacity):
    print(f"=== {title} (capacity {capacity}) ===")
    print("Items (id, weight, value, ratio):", [(i, w, v, round(v / w, 2)) for i, w, v in items])
    g_val, g_sel, g_steps, g_ops = fractional_greedy(items, capacity)
    print("Fractional greedy decision sequence:")
    for s in g_steps:
        print("  ", s)
    print(f"Fractional greedy value = {g_val:.1f}, selection = {g_sel}, ops = {g_ops}")
    gz_val, gz_sel, gz_steps, gz_ops = greedy_01(items, capacity)
    print("0/1 greedy (ratio rule) sequence:")
    for s in gz_steps:
        print("  ", s)
    d_val, d_sel, d_ops = knapsack_dp(items, capacity)
    print(f"0/1 greedy value = {gz_val}, selection = {gz_sel}")
    print(f"0/1 DP optimum   = {d_val}, selection = {d_sel}, ops = {d_ops}")
    print("Greedy == DP optimum?", gz_val == d_val, "\n")


def random_instance(rng, n, max_w, max_v):
    return [(f"I{k}", rng.randint(1, max_w), rng.randint(1, max_v)) for k in range(n)]
```

```
def quality_study(trials=2000, seed=7):
    rng = random.Random(seed)
    worse, worst_ratio, total_ratio, improved_worse = 0, 1.0, 0.0, 0
    for _ in range(trials):
        items = random_instance(rng, 8, 20, 60)
        cap = sum(w for _, w, _ in items) // 3
        opt = knapsack_dp(items, cap)[0]
        greedy = greedy_01(items, cap)[0]
        best_single = max((v for _, w, v in items if w <= cap), default=0)
        improved = max(greedy, best_single)
        ratio = greedy / opt if opt else 1.0
        total_ratio += ratio
        worst_ratio = min(worst_ratio, ratio)
        worse += greedy < opt
        improved_worse += improved < opt
    print(f'Quality over {trials} random instances (n=8): greedy suboptimal in {worse} ({100 *
worse / trials:.1f}%), '
          f'mean greedy/opt = {total_ratio / trials:.4f}, worst greedy/opt = {worst_ratio:.3f}, '
          f"'greedy or best single item' suboptimal in {improved_worse}")


if __name__ == "__main__":
    normal = [("P1", 5, 50), ("P2", 10, 80), ("P3", 15, 90), ("P4", 20, 60)]
    show_case("2.1 Normal case", normal, 30)
    show_case("2.1 Normal case, fractional split needed", normal, 40)
    adversarial = [("A", 6, 30), ("B", 5, 24), ("C", 5, 24), ("D", 4, 12)]
    show_case("2.2 Adversarial 0/1 instance", adversarial, 10)

    print("=== 2.3 Engineering comparison ===")
    rng = random.Random(11)
    print(f"{'n':>5} {'W':>7} | {'greedy cmp':>10} {'greedy mem':>10} | {'DP cells':>10} {'DP
mem':>10}")
    for n, cap in [(10, 100), (50, 1000), (200, 10000), (200, 100000)]:
        items = random_instance(rng, n, 100, 500)
        _, _, _, g = greedy_01(items, cap)
        _, _, d = knapsack_dp(items, cap)
        print(f"{n:>5} {cap:>7} | {g['comparisons']:>10} {g['memory_cells']:>10} |
{d['cell_updates']:>10} {d['memory_cells']:>10}")
    quality_study()
```

Normal case. The dataset is P1 (w 5, v 50), P2 (w 10, v 80), P3 (w 15, v 90), P4 (w 20, v 60), with capacity 30. The ratios are 10, 8, 6 and 3. **Expected:** greedy takes P1, P2 and P3 in that order;

their weights sum exactly to 30, so no item has to be split and greedy should equal the DP optimum of 220.

Output 3 – cases 2.1 (normal) and 2.2 (adversarial)

```
=== 2.1 Normal case (capacity 30) ===
Items (id, weight, value, ratio): [('P1', 5, 50, 10.0), ('P2', 10, 80, 8.0), ('P3', 15, 90, 6.0), ('P4', 20, 60, 3.0)]
Fractional greedy decision sequence:
  take 1.00 of P1 (ratio 10.00) -> value +50.0, capacity left 25.0
  take 1.00 of P2 (ratio 8.00) -> value +80.0, capacity left 15.0
  take 1.00 of P3 (ratio 6.00) -> value +90.0, capacity left 0.0
Fractional greedy value = 220.0, selection = [('P1', 1.0), ('P2', 1.0), ('P3', 1.0)], ops = {'comparisons': 3, 'decisions': 3, 'memory_cells': 4}
0/1 greedy (ratio rule) sequence:
  take P1 (w=5, v=50, ratio 10.00), capacity left 25
  take P2 (w=10, v=80, ratio 8.00), capacity left 15
  take P3 (w=15, v=90, ratio 6.00), capacity left 0
  skip P4 (w=20 > capacity left 0)
0/1 greedy value = 220, selection = ['P1', 'P2', 'P3']
0/1 DP optimum   = 220, selection = ['P1', 'P2', 'P3'], ops = {'cell_updates': 124, 'memory_cells': 155}
Greedy == DP optimum? True

=== 2.1 Normal case, fractional split needed (capacity 40) ===
Items (id, weight, value, ratio): [('P1', 5, 50, 10.0), ('P2', 10, 80, 8.0), ('P3', 15, 90, 6.0), ('P4', 20, 60, 3.0)]
Fractional greedy decision sequence:
  take 1.00 of P1 (ratio 10.00) -> value +50.0, capacity left 35.0
  take 1.00 of P2 (ratio 8.00) -> value +80.0, capacity left 25.0
  take 1.00 of P3 (ratio 6.00) -> value +90.0, capacity left 10.0
  take 0.50 of P4 (ratio 3.00) -> value +30.0, capacity left 0.0
Fractional greedy value = 250.0, selection = [('P1', 1.0), ('P2', 1.0), ('P3', 1.0), ('P4', 0.5)], ops = {'comparisons': 3, 'decisions': 4, 'memory_cells': 4}
0/1 greedy (ratio rule) sequence:
  take P1 (w=5, v=50, ratio 10.00), capacity left 35
  take P2 (w=10, v=80, ratio 8.00), capacity left 25
  take P3 (w=15, v=90, ratio 6.00), capacity left 10
  skip P4 (w=20 > capacity left 10)
0/1 greedy value = 220, selection = ['P1', 'P2', 'P3']
0/1 DP optimum   = 220, selection = ['P1', 'P2', 'P3'], ops = {'cell_updates': 164, 'memory_cells': 205}
Greedy == DP optimum? True

=== 2.2 Adversarial 0/1 instance (capacity 10) ===
Items (id, weight, value, ratio): [('A', 6, 30, 5.0), ('B', 5, 24, 4.8), ('C', 5, 24, 4.8), ('D', 4, 12, 3.0)]
```

```
Fractional greedy decision sequence:
  take 1.00 of A (ratio 5.00) -> value +30.0, capacity left 4.0
  take 0.80 of B (ratio 4.80) -> value +19.2, capacity left 0.0
Fractional greedy value = 49.2, selection = [('A', 1.0), ('B', 0.8)], ops = {'comparisons': 3,
'decisions': 2, 'memory_cells': 4}
0/1 greedy (ratio rule) sequence:
  take A (w=6, v=30, ratio 5.00), capacity left 4
  skip B (w=5 > capacity left 4)
  skip C (w=5 > capacity left 4)
  take D (w=4, v=12, ratio 3.00), capacity left 0
0/1 greedy value = 42, selection = ['A', 'D']
0/1 DP optimum   = 48, selection = ['B', 'C'], ops = {'cell_updates': 44, 'memory_cells': 55}
Greedy == DP optimum? False
```

The decision sequence shows three takes ("take 1.00 of P1 … P2 … P3") and then the capacity is exhausted. Fractional greedy, 0/1 greedy and the DP all return 220 with the same items. The ratio rule works here because the capacity is consumed exactly. With capacity 40 the fractional greedy takes half of P4 and reaches 250, while the 0/1 optimum is still 220. This shows that the fractional answer is an upper bound on the 0/1 answer, and that the greedy rule is provably optimal only for the fractional formulation.

2.2 Breaking the Greedy Intuition

Student-designed instance (capacity 10): A (w 6, v 30, ratio 5.0), B (w 5, v 24, ratio 4.8), C (w 5, v 24, ratio 4.8), D (w 4, v 12, ratio 3.0). The ratio rule ranks A first. **Result:** 0/1 greedy takes A (capacity left 4), skips B and C because each weighs 5, and then takes D, for a value of **42** with items {A, D}. The DP optimum is **48** with items {B, C}, so GreedyResult ≠ DPOptimum is demonstrated by the program itself ("Greedy == DP optimum? False").

The first greedy commitment that prevents the optimum is taking item A at step 1. It has the best ratio but its weight of 6 leaves a remainder of 4, a capacity that neither B nor C can use. The two items B and C fill the knapsack exactly (5 + 5 = 10), so no capacity is wasted. The ratio rule measures value per unit of weight but ignores the granularity of the items: capacity left over is lost value, and the 0/1 problem cannot repair it by splitting an item. The fractional greedy on the same data reaches 49.2 (A plus 0.8 of B), which exceeds both 0/1 results, again because splitting is allowed.

2.3 Engineering Comparison

The two solutions were instrumented with counters instead of relying on execution time. The greedy solution counts comparisons made while sorting by ratio and decisions made; the DP counts every table-cell update and the number of cells stored. Four random instances (n = 10 to 200, capacities from 100 to 100000) were generated with a fixed seed, and quality was measured over 2000 random instances with n = 8. The results are in the last part of Output 4.

Output 4 – operation counts and quality study

```
=== 2.3 Engineering comparison ===
```

```
   n      W | greedy cmp greedy mem |   DP cells     DP mem
  10    100 |        23         10 |       1010       1111
  50   1000 |       220         50 |      50050      51051
 200  10000 |      1262        200 |    2000200    2010201
 200 100000 |      1259        200 |   20000200   20100201
```

Quality over 2000 random instances (n=8): greedy suboptimal in 662 (33.1%), mean greedy/opt = 0.9759, worst greedy/opt = 0.686, 'greedy or best single item' suboptimal in 662

Criterion	Greedy (ratio rule)	Dynamic programming
Computational work	About n log n comparisons: 23, 220 and 1262 for n = 10, 50 and 200. Independent of the capacity W (1262 against 1259 when W grew tenfold).	n × (W+1) cell updates: 1,010; 50,050; 2,000,200; and 20,000,200 when W rose from 10,000 to 100,000. Grows linearly with the numeric value of W.
Memory	O(n): 200 entries for n = 200.	O(n·W): 2,010,201 cells for n = 200 and W = 10,000, and 20,100,201 for W = 100,000, about 10,000 to 100,000 times more.
Solution quality	Exact for the fractional problem. For 0/1: suboptimal in 662 of 2000 random cases (33.1%), mean 97.6% of the optimum, worst case 68.6%.	Always the exact optimum.
Optimality guarantee	Proved for fractional knapsack only; no guarantee for 0/1.	Guaranteed for 0/1 knapsack (integer weights).

A point worth defending in a viva: the DP is called “pseudo-polynomial” because its work depends on the numeric value of the capacity, not just on the number of items. The counters show it directly: multiplying W by ten multiplied the DP work and memory by ten, and left the greedy work unchanged.

2.4 Changed Requirement – “Return the Decision Extremely Quickly”

Conclusion: select the greedy ratio strategy for the live decision, and keep the DP for offline verification. The reasoning is based on the measured behaviour. First, the greedy decision costs only a sort and one pass: 1262 comparisons for 200 items regardless of capacity, whereas the DP needed 2 million to 20 million cell updates and up to 20 million cells of memory for the same items, and would grow further if capacities were measured in finer units (for example grams instead of kilograms). Second, near-optimal is acceptable, and the quality study shows greedy reaches on average 97.6% of the optimum, and is exactly optimal in about two out of three random instances (1338 of 2000). Third, the same study exposes the risk that management must accept: the worst observed case was 68.6% of the optimum, so a guarantee of near-optimality is not available, only typical behaviour. I also tested a common repair, taking the better of the greedy result and the best single item; on these random instances it did not reduce the number of suboptimal cases (still 662), so it would add code without evidence of benefit and I do not

recommend it here. If management later requires a guaranteed quality bound, the evidence says the DP, or a DP over scaled values, would be needed, at a cost that must be justified against the time limit.

Task 3 – Routing Engine under Changing Network Conditions (10 Marks)

Graph definition and implementation

The logistics network has **8 vertices (A to H) and 13 directed weighted edges**: A→B 4, A→C 2, C→B 1, B→D 5, C→E 8, B→E 9, D→F 3, D→G 9, E→F 6, E→G 7, F→G 2, F→H 4, G→H 2. Source A is used throughout. The program implements Dijkstra with a binary heap and an explicit finalised set (a vertex is never updated once finalised, which is the real assumption behind Dijkstra's algorithm), Bellman-Ford with a final |V|-th pass for cycle detection, Floyd's algorithm for all-pairs distances, and Warshall's algorithm for reachability. Paths are rebuilt by following predecessor links backwards from the target.

Listing 3 – task3_routing.py (source code)

```
"""Task 3 - Routing engine: Dijkstra, Bellman-Ford, negative-cycle detection, Floyd, Warshall."""
import heapq

INF = float("inf")
NODES = list("ABCDEFGH")
BASE_EDGES = [("A", "B", 4), ("A", "C", 2), ("C", "B", 1), ("B", "D", 5), ("C", "E", 8), ("B", "E", 9),
              ("D", "F", 3), ("D", "G", 9), ("E", "F", 6), ("E", "G", 7), ("F", "G", 2), ("F", "H", 4), ("G", "H", 2)]
NEGATIVE_EDGE = ("E", "D", -5)      # Stage B: a late vertex improves an early, already-finalised one
CYCLE_EDGE = ("G", "F", -5)         # Stage C: F->G (2) + G->F (-5) = -3 reachable cycle


def adjacency(edges):
    graph = {v: [] for v in NODES}
    for u, v, w in edges:
        graph[u].append((v, w))
    return graph


def dijkstra(graph, source):
    dist = {v: INF for v in graph}
    pred = {v: None for v in graph}
    dist[source], finalised, order = 0, set(), []
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if u in finalised:
```

```
            continue
        finalised.add(u)                    # Dijkstra's assumption: u's distance is now permanent
        order.append(u)
        for v, w in graph[u]:
            if v not in finalised and d + w < dist[v]:
                dist[v], pred[v] = d + w, u
                heapq.heappush(heap, (dist[v], v))
    return dist, pred, order


def bellman_ford(nodes, edges, source):
    dist = {v: INF for v in nodes}
    pred = {v: None for v in nodes}
    dist[source] = 0
    for _ in range(len(nodes) - 1):
        changed = False
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v], pred[v], changed = dist[u] + w, u, True
        if not changed:
            break
    for u, v, w in edges:                   # V-th pass: any further improvement proves a negative cycle
        if dist[u] + w < dist[v]:
            pred[v] = u
            return dist, pred, extract_cycle(pred, v, len(nodes))
    return dist, pred, None


def extract_cycle(pred, start, count):
    for _ in range(count):                  # walk back far enough to be certainly inside the cycle
        start = pred[start]
    cycle, node = [start], pred[start]
    while node != start:
        cycle.append(node)
        node = pred[node]
    return cycle[::-1] + [cycle[-1]]


def path_to(pred, target):
    path = []
    while target is not None:
        path.append(target)
        target = pred[target]
    return "->".join(reversed(path))
```

```
def floyd(nodes, edges):
    dist = {u: {v: (0 if u == v else INF) for v in nodes} for u in nodes}
    for u, v, w in edges:
        dist[u][v] = min(dist[u][v], w)
    for k in nodes:
        for i in nodes:
            for j in nodes:
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist


def warshall(nodes, edges):
    reach = {u: {v: u == v for v in nodes} for u in nodes}
    for u, v, _ in edges:
        reach[u][v] = True
    for k in nodes:
        for i in nodes:
            for j in nodes:
                reach[i][j] = reach[i][j] or (reach[i][k] and reach[k][j])
    return reach


def print_distances(title, dist, pred, order=None):
    print(title)
    if order:
        print("Finalisation order:", " ".join(order))
    for v in NODES:
        d = "inf" if dist[v] == INF else dist[v]
        print(f"  {v}: dist={d!s:>4} pred={pred[v]!s:>4} path={path_to(pred, v)}")


def print_matrix(title, matrix, cell):
    print(title)
    print("     " + " ".join(f"{v:>4}" for v in NODES))
    for u in NODES:
        print(f"  {u}  " + " ".join(f"{cell(matrix[u][v]):>4}" for v in NODES))


if __name__ == "__main__":
    print(f"Graph: {len(NODES)} vertices, {len(BASE_EDGES)} directed edges:
{BASE_EDGES}\n")
```

```
    d, p, order = dijkstra(adjacency(BASE_EDGES), "A")
    print_distances("STAGE A - Dijkstra from A", d, p, order)

    edges_b = BASE_EDGES + [NEGATIVE_EDGE]
    print("\nSTAGE B - negative edge E->D (-5) added")
    d, p, order = dijkstra(adjacency(edges_b), "A")
    print_distances("Dijkstra (finalised-set version):", d, p, order)
    bf_d, bf_p, cyc = bellman_ford(NODES, edges_b, "A")
    print_distances("Bellman-Ford:", bf_d, bf_p)
    print("Vertices where Dijkstra is wrong:", [v for v in NODES if d[v] != bf_d[v]])

    edges_c = edges_b + [CYCLE_EDGE]
    print("\nSTAGE C - negative cycle G->F (-5) added")
    c_d, c_p, cyc = bellman_ford(NODES, edges_c, "A")
    print("Negative cycle detected:" , cyc and " -> ".join(cyc), "| cycle weight =",
          sum(w for u, v, w in edges_c if (u, v) in set(zip(cyc, cyc[1:]))))
    naive = {v: INF for v in NODES}; naive["A"] = 0
    for _ in range(len(NODES) - 1):
        for u, v, w in edges_c:
            naive[v] = min(naive[v], naive[u] + w)
    print("Misleading V-1 pass result (what a naive run would report):", {v: naive[v] for v in
"FGH"})
    fl_c = floyd(NODES, edges_c)
    print("Floyd diagonal on cyclic graph:", {v: fl_c[v][v] for v in NODES if fl_c[v][v] < 0})

    print("\nSTAGE D - all pairs on the Stage B graph")
    show = lambda x: "inf" if x == INF else str(x)
    print_matrix("Floyd shortest-distance matrix:", floyd(NODES, edges_b), show)
    print_matrix("Warshall reachability matrix (1 = reachable):", warshall(NODES, edges_b),
lambda x: "1" if x else "0")
```

Stage A – Normal Routing with Dijkstra

Output 5 – Stage A

```
Graph: 8 vertices, 13 directed edges: [('A', 'B', 4), ('A', 'C', 2), ('C', 'B', 1), ('B', 'D', 5), ('C', 'E', 8),
('B', 'E', 9), ('D', 'F', 3), ('D', 'G', 9), ('E', 'F', 6), ('E', 'G', 7), ('F', 'G', 2), ('F', 'H', 4), ('G', 'H', 2)]

STAGE A - Dijkstra from A
Finalisation order: A C B D E F G H
  A: dist=   0 pred=None path=A
  B: dist=   3 pred=   C path=A->C->B
  C: dist=   2 pred=   A path=A->C
  D: dist=   8 pred=   B path=A->C->B->D
  E: dist=  10 pred=   C path=A->C->E
  F: dist=  11 pred=   D path=A->C->B->D->F
```

```
  G: dist=  13 pred=   F path=A->C->B->D->F->G
  H: dist=  15 pred=   F path=A->C->B->D->F->H
```

The vertices are finalised in the order A, C, B, D, E, F, G, H, which is the order of non-decreasing distance (0, 2, 3, 8, 10, 11, 13, 15). The shortest route to H is A→C→B→D→F→H with cost 15; the predecessor of H is F, of F is D, of D is B, of B is C and of C is A. Vertex G (13) is reached through F rather than directly from D (17) or E (17), a small example of Dijkstra preferring a longer path in edges that is shorter in cost.

Stage B – Introducing a Negative Edge

Prediction recorded before the run. I add the edge E→D with weight −5. D is finalised (distance 8) before E (distance 10). When E is finalised later, the edge E→D offers 10 − 5 = 5, which is better, but Dijkstra never revisits a finalised vertex. I therefore predict that **Dijkstra will no longer be valid**: the route to D will remain A→C→B→D (8) instead of the true A→C→E→D (5), and every route that passes through D (to F, G and H) will be wrong by the same 3 units. Bellman-Ford does not rely on finalisation and should return D = 5, F = 8, G = 10 and H = 12. **Design of the test:** the negative edge was chosen so that its head D is finalised before its tail E and D cannot reach E, so no negative cycle exists. An arbitrary negative edge, for example into a leaf vertex, would not expose any difference, which is why this edge was selected carefully.

Output 6 – Stage B comparison

```
STAGE B - negative edge E->D (-5) added
Dijkstra (finalised-set version):
Finalisation order: A C B D E F G H
  A: dist=   0 pred=None path=A
  B: dist=   3 pred=   C path=A->C->B
  C: dist=   2 pred=   A path=A->C
  D: dist=   8 pred=   B path=A->C->B->D
  E: dist=  10 pred=   C path=A->C->E
  F: dist=  11 pred=   D path=A->C->B->D->F
  G: dist=  13 pred=   F path=A->C->B->D->F->G
  H: dist=  15 pred=   F path=A->C->B->D->F->H
Bellman-Ford:
  A: dist=   0 pred=None path=A
  B: dist=   3 pred=   C path=A->C->B
  C: dist=   2 pred=   A path=A->C
  D: dist=   5 pred=   E path=A->C->E->D
  E: dist=  10 pred=   C path=A->C->E
  F: dist=   8 pred=   D path=A->C->E->D->F
  G: dist=  10 pred=   F path=A->C->E->D->F->G
  H: dist=  12 pred=   F path=A->C->E->D->F->H
Vertices where Dijkstra is wrong: ['D', 'F', 'G', 'H']
```

Observation. The run confirms the prediction exactly. Dijkstra returns the unchanged table (D = 8, F = 11, G = 13, H = 15) and silently reports wrong answers; it gives no error. Bellman-Ford returns D = 5 (A→C→E→D), F = 8, G = 10 and H = 12, and the program lists the vertices where Dijkstra is wrong: D, F, G and H. The danger is practical: the faulty route is not just longer, it is presented as optimal.

Stage C – Production Failure: Reachable Negative-Weight Cycle

The edge G→F with weight −5 is added. Together with F→G (2) it forms the cycle F→G→F of weight −3, reachable from A. A shortest path no longer exists, because every traversal of the cycle reduces the distance by 3.

How the implementation detects it. In a graph with V vertices and no negative cycle, every shortest path has at most V−1 edges, so V−1 relaxation passes are enough and a V-th pass must change nothing. The program performs the V-th pass; if any edge can still be relaxed, a negative cycle is reachable and the function returns the cycle instead of distances. To report the cycle itself, the program follows the predecessor links V times (guaranteed to land inside the cycle) and then walks around it once.

Output 7 – Stage C

```
STAGE C - negative cycle G->F (-5) added
Negative cycle detected: F -> G -> F | cycle weight = -3
Misleading V-1 pass result (what a naive run would report): {'F': -10, 'G': -5, 'H': -3}
Floyd diagonal on cyclic graph: {'F': -3, 'G': -6}
```

The program reports the cycle F → G → F with weight −3. The line “Misleading V−1 pass result” shows what would have been returned without the check: F = −10, G = −5, H = −3. These numbers have no meaning, since another pass would lower them again. The Floyd diagonal check gives an independent confirmation: dist[F][F] = −3 and dist[G][G] = −6 are negative, and a negative diagonal entry always signals a negative cycle through that vertex.

Stage D – All-Pairs Cost and Reachability

The analytics team needs the cost between every pair and whether each location can reach another. Floyd’s algorithm runs on the Stage B graph (which contains the negative edge but no cycle) and Warshall’s algorithm computes the boolean closure of the same graph. Both use the triple loop over an intermediate vertex k, i, j; Floyd tests dist[i][k] + dist[k][j] < dist[i][j], and Warshall replaces the arithmetic by reach[i][k] AND reach[k][j].

Output 8 – Floyd distance matrix and Warshall reachability matrix

```
STAGE D - all pairs on the Stage B graph
Floyd shortest-distance matrix:
     A   B   C   D   E   F   G   H
 A   0   3   2   5  10   8  10  12
 B inf   0 inf   4   9   7   9  11
 C inf   1   0   3   8   6   8  10
 D inf inf inf   0 inf   3   5   7
 E inf inf inf  -5   0  -2   0   2
```

```
F   inf  inf  inf  inf  inf   0    2    4
G   inf  inf  inf  inf  inf  inf   0    2
H   inf  inf  inf  inf  inf  inf  inf   0
```

Warshall reachability matrix (1 = reachable):

```
     A   B   C   D   E   F   G   H
A    1   1   1   1   1   1   1   1
B    0   1   0   1   1   1   1   1
C    0   1   1   1   1   1   1   1
D    0   0   0   1   0   1   1   1
E    0   0   0   1   1   1   1   1
F    0   0   0   0   0   1   1   1
G    0   0   0   0   0   0   1   1
H    0   0   0   0   0   0   0   1
```

Interpretation. Row A of the Floyd matrix (0, 3, 2, 5, 10, 8, 10, 12) equals the Bellman-Ford result, which cross-checks the two algorithms. Row E shows the effect of the negative edge: E reaches D at cost −5 and F at −2 (E→D→F), cheaper than the direct edge E→F of 6. Infinity entries correspond exactly to zeros in the Warshall matrix, so the two matrices agree: A reaches every vertex, H reaches only itself, and no vertex can go back to A. The cost is $O(V^3) = 512$ inner steps for V = 8 for each algorithm. Floyd would be the wrong tool for the graph of Stage C, where the negative diagonal entries mean the matrix is not meaningful.

Task 4 – Smart Manufacturing Optimisation: Assembly-Line Scheduling (7 Marks)

4.1 Building the Optimisation Engine

Model. There are two lines and n stations per line. a[l][i] is the processing time at station i on line l; transfer[l][i] is the extra time to leave line l after station i and enter the other line at station i+1; e[l] and x[l] are the entry and exit times. Let finish[l][i] be the least time to complete station i on line l. **Recurrence:** finish[l][1] = e[l] + a[l][1]; for i > 1, finish[l][i] = min(finish[l][i−1] + a[l][i] (stay), finish[other][i−1] + transfer[other][i−1] + a[l][i] (transfer)). The answer is min over l of finish[l][n] + x[l]. The optimal substructure is that the fastest way to reach a station on a line depends only on the fastest ways to reach the previous station on either line. The program stores which choice won in came_from and walks back from the best exit to reconstruct the line used at each station, so the route and all transfer points are known. The work is O(n) and the memory is O(n).

Test data. Line 1 times: 7, 9, 3, 4, 8, 4. Line 2 times: 8, 5, 6, 4, 5, 7. Transfers from Line 1 after stations 1 to 5: 2, 3, 1, 3, 4; from Line 2: 2, 1, 2, 2, 1. Entry times 2 (Line 1) and 4 (Line 2); exit times 3 and 6.

Listing 4 – task4_assembly.py (source code)

```
"""Task 4 - Two-line assembly scheduling by dynamic programming."""
import copy

BASE = {
   "station_time": [[7, 9, 3, 4, 8, 4], [8, 5, 6, 4, 5, 7]],   # processing time per station, per line
```

```
    "transfer": [[2, 3, 1, 3, 4], [2, 1, 2, 2, 1]],          # transfer[l][i]: leave line l after station i for the other line
    "entry": [2, 4],
    "exit": [3, 6],
}


def validate(data):
    n = len(data["station_time"][0])
    if n == 0 or len(data["station_time"][1]) != n:
        raise ValueError("both lines need the same, non-zero number of stations")
    if any(len(row) != n - 1 for row in data["transfer"]):
        raise ValueError("each transfer list needs (stations - 1) entries")
    if any(t < 0 for row in data["station_time"] + data["transfer"] for t in row):
        raise ValueError("times must be non-negative")


def schedule(data):
    """finish[l][i] = least time to complete station i on line l. Returns trace rows and the optimal route."""
    validate(data)
    a, t, e, x = data["station_time"], data["transfer"], data["entry"], data["exit"]
    n = len(a[0])
    finish = [[e[l] + a[l][0]] + [0] * (n - 1) for l in range(2)]
    came_from = [[None] * n for _ in range(2)]                # which line the product arrived from
    trace = []
    for i in range(1, n):
        row = {"station": i + 1}
        for l in range(2):
            other = 1 - l
            stay = finish[l][i - 1] + a[l][i]
            move = finish[other][i - 1] + t[other][i - 1] + a[l][i]
            finish[l][i] = min(stay, move)
            came_from[l][i] = l if stay <= move else other
            row[l] = (stay, move, "stay" if stay <= move else "transfer")
        trace.append(row)
    totals = [finish[l][n - 1] + x[l] for l in range(2)]
    last_line = 0 if totals[0] <= totals[1] else 1
    lines = [last_line]
    for i in range(n - 1, 0, -1):
        lines.append(came_from[lines[-1]][i])
    lines.reverse()                                           # lines[i] = line used at station i
    return {"best": totals[last_line], "lines": lines, "finish": finish, "trace": trace, "exit_totals": totals}
```

```
def print_result(label, data):
    r = schedule(data)
    a, e, finish = data["station_time"], data["entry"], r["finish"]
    print(f"=== {label} ===")
    print(f"Station 1: Line1 = {finish[0][0]} (entry {e[0]} + {a[0][0]}), Line2 = {finish[1][0]}
(entry {e[1]} + {a[1][0]})")
    print("Stn | L1 stay  L1 move  pick     | L2 stay  L2 move  pick")
    for row in r["trace"]:
        s1, m1, c1 = row[0]
        s2, m2, c2 = row[1]
        print(f"{row['station']:>3} | {s1:>7} {m1:>8}  {c1:<9}| {s2:>7} {m2:>8}  {c2}")
    print(f"Exit totals: Line1 {r['exit_totals'][0]}, Line2 {r['exit_totals'][1]}")
    print("Entry")
    prev = None
    for i, l in enumerate(r["lines"]):
        print(" v Transfer" if prev is not None and l != prev else " v")
        print(f"Line {l + 1} - Station {i + 1}")
        prev = l
    print(" v\nExit")
    transfers = [(i + 1, r["lines"][i] + 1, r["lines"][i + 1] + 1) for i in range(len(r["lines"]) - 1) if
r["lines"][i] != r["lines"][i + 1]]
    print(f"Transfer points (after station, from line, to line): {transfers}")
    print(f"Minimum Production Time = {r['best']}\n")
    return r


if __name__ == "__main__":
    base = print_result("Base case", BASE)
    heavy = copy.deepcopy(BASE)
    heavy["transfer"][1][1] += 20
    print_result("4.3 transfer time raised by 20 (Line 2 -> Line 1 after station 2: 1 -> 21)", heavy)
    slow = copy.deepcopy(BASE)
    slow["station_time"][1][1] = 25
    print_result("4.4 disruption: Line 2 station 2 slows 5 -> 25", slow)
    try:
        schedule({"station_time": [[1, 2], [1]], "transfer": [[1], []], "entry": [0, 0], "exit": [0, 0]})
    except ValueError as err:
        print("Invalid input rejected:", err)
```

Output 9 – base case, tasks 4.1 and 4.2

```
=== Base case ===
Station 1: Line1 = 9 (entry 2 + 7), Line2 = 12 (entry 4 + 8)
```

```
Stn | L1 stay  L1 move  pick     | L2 stay  L2 move  pick
  2 |     18      23  stay     |     17      16  transfer
  3 |     21      20  transfer |     22      27  stay
  4 |     24      28  stay     |     26      25  transfer
  5 |     32      35  stay     |     30      32  stay
  6 |     36      35  transfer |     37      43  stay
Exit totals: Line1 38, Line2 43
Entry
 v
Line 1 - Station 1
 v Transfer
Line 2 - Station 2
 v Transfer
Line 1 - Station 3
 v Transfer
Line 2 - Station 4
 v
Line 2 - Station 5
 v Transfer
Line 1 - Station 6
 v
Exit
Transfer points (after station, from line, to line): [(1, 1, 2), (2, 2, 1), (3, 1, 2), (5, 2, 1)]
Minimum Production Time = 38
```

Result. The minimum production time is **38**. The route is Line 1 – Station 1, transfer to Line 2 – Station 2, transfer to Line 1 – Station 3, transfer to Line 2 – Stations 4 and 5, transfer to Line 1 – Station 6, exit. The transfer points are after stations 1, 2, 3 and 5. The route is printed as an arrow diagram, as the task requests, and not only as a number.

4.2 Exposing the Decision Process

The trace table in Output 9 lists, for every station from 2 to 6, the cost of reaching each line by staying and by transferring, and which one was selected. For example, at station 2 on Line 2, staying costs 12 + 5 = 17 while transferring from Line 1 costs 9 + 2 + 5 = 16, so “transfer” wins by one unit. At station 3 on Line 1, staying costs 18 + 3 = 21, while transferring from Line 2 costs 16 + 1 + 3 = 20, so the transfer is selected. At the end the exit totals are 38 (Line 1) and 43 (Line 2). Every cell of the table uses only values of the previous row, which is the optimal substructure made visible during execution: the best route to station i is built from the best routes to station i−1.

4.3 Predict Before You Execute

Change chosen: the transfer time from Line 2 to Line 1 after station 2 is increased from 1 to 21. **Prediction recorded first:** (1) the optimal route will change, because this transfer is used in the current optimal route; (2) the decision most likely to be affected is the one at station 3, where the

product currently leaves Line 2 for Line 1, so it should now stay on Line 2; (3) the total time will increase, but by clearly less than 20, because the alternative routes that avoid the expensive transfer are only slightly slower.

Output 10 – experiments 4.3 and 4.4 and the invalid-input test

```
=== 4.3 transfer time raised by 20 (Line 2 -> Line 1 after station 2: 1 -> 21) ===
Station 1: Line1 = 9 (entry 2 + 7), Line2 = 12 (entry 4 + 8)
Stn | L1 stay  L1 move  pick     | L2 stay  L2 move  pick
  2 |     18       23  stay     |     17       16  transfer
  3 |     21       40  stay     |     22       27  stay
  4 |     25       28  stay     |     26       26  stay
  5 |     33       36  stay     |     31       33  stay
  6 |     37       36  transfer |     38       44  stay
Exit totals: Line1 39, Line2 44
Entry
 v
Line 1 - Station 1
 v Transfer
Line 2 - Station 2
 v
Line 2 - Station 3
 v
Line 2 - Station 4
 v
Line 2 - Station 5
 v Transfer
Line 1 - Station 6
 v
Exit
Transfer points (after station, from line, to line): [(1, 1, 2), (5, 2, 1)]
Minimum Production Time = 39

=== 4.4 disruption: Line 2 station 2 slows 5 -> 25 ===
Station 1: Line1 = 9 (entry 2 + 7), Line2 = 12 (entry 4 + 8)
Stn | L1 stay  L1 move  pick     | L2 stay  L2 move  pick
  2 |     18       23  stay     |     37       36  transfer
  3 |     21       40  stay     |     42       27  transfer
  4 |     25       33  stay     |     31       26  transfer
  5 |     33       36  stay     |     31       33  stay
  6 |     37       36  transfer |     38       44  stay
Exit totals: Line1 39, Line2 44
Entry
 v
Line 1 - Station 1
 v
```

```
Line 1 - Station 2
 v
Line 1 - Station 3
 v Transfer
Line 2 - Station 4
 v
Line 2 - Station 5
 v Transfer
Line 1 - Station 6
 v
Exit
Transfer points (after station, from line, to line): [(3, 1, 2), (5, 2, 1)]
Minimum Production Time = 39

Invalid input rejected: both lines need the same, non-zero number of stations
```

Observed: the route became Line 1 (station 1), Line 2 (stations 2 to 5), Line 1 (station 6), with only two transfers, and the minimum time rose from 38 to **39**. All three predictions were correct: the route changed, the affected decision was at station 3 (Line 2 now stays, 22 against 27 by transferring), and the increase was only 1 unit, not 20. At station 4 on Line 2 the two options tie at 26 and the program keeps the product on its current line, which is a deliberate tie-break rule (prefer staying) and makes the result deterministic.

4.4 Production Disruption Challenge

A machine at station 2 on Line 2 slows down: its processing time rises from 5 to 25. **Results:** (a) the optimal route changes; (b) yes, production is moved to the alternative line, so the product now stays on Line 1 for stations 1, 2 and 3; (c) the route changes at station 2, where the earlier route moved to Line 2, and then transfers to Line 2 only after station 3 (for stations 4 and 5) before returning to Line 1; (d) the minimum time increases from 38 to 39. **Why the new route is optimal:** station 2 on Line 2 now costs 36 at best (stay 37, transfer 36), whereas station 2 on Line 1 costs 18. Avoiding the slow machine costs only one extra unit overall, because Line 1 is competitive at stations 2 and 3 (9 and 3 minutes). The DP compares all alternatives at every station, so it shows that no route is faster than 39: the best of the other routes passing through the slow station is slower.

Failure test: an input in which the two lines have a different number of stations is rejected with a ValueError (“both lines need the same, non-zero number of stations”), shown at the end of Output 10.

4.5 Engineering Extension – Three Assembly Lines

DP state. The state finish[l][i] would range over $l \in \{1, 2, 3\}$, so the table grows from $2 \times n$ to $3 \times n$ entries; the meaning (least time to complete station i on line l) is unchanged, so the optimal substructure still holds. **Transition logic.** Instead of choosing between “stay” and one transfer, each line now has three candidates: stay, transfer from either of the two other lines. The recurrence becomes finish[l][i] = a[l][i] + min over k of (finish[k][i−1] + (0 if k = l else transfer[k][l][i−1])). The transfer data can no longer be a single number per line per stage: it becomes a 3×3 matrix

per gap between stations, because moving from line 1 to line 3 may cost a different amount than going from 3 to 1 or from 1 to 2. If the problem allowed several hops between two stations, the transfer matrix would first have to be closed by a shortest-path computation (Floyd); in the model of the assignment one transfer per gap is allowed, so this is not needed. Entry and exit arrays gain a third entry. **Route reconstruction.** came_from[l][i] must store the identity of the best previous line, one of three values rather than a stay/transfer flag, and the backward walk is unchanged in structure: pick the line with the best finish plus exit time, then repeatedly jump to came_from. **Cost.** With L lines the work is $O(n \cdot L^2)$ and the memory $O(n \cdot L)$, so three lines multiply the work by 9/4 relative to two lines, but the algorithm remains linear in the number of stations. The trace table would need three candidate columns for each line.

Task 5 – Three Greedy Engineers, One Network: Kruskal, Prim and Borůvka (7 Marks)

Network, implementation and construction traces

The network has 8 facilities (A to H) and 14 undirected edges with distinct weights: A–B 4, A–C 3, B–C 5, B–D 7, C–D 6, C–E 8, D–E 2, D–F 9, E–F 10, E–G 11, F–G 1, F–H 12, G–H 13, B–H 14. All three algorithms run on this one graph and print how their solution is built. Kruskal and Borůvka share a disjoint-set (union-find) structure with path compression; Prim uses a binary heap of frontier edges. Borůvka breaks ties by a fixed total order of edges, which is needed for correctness when weights are equal (see below).

Listing 5 – task5_mst.py (source code)

```
"""Task 5 - Kruskal, Prim and Boruvka on the same network, with construction traces and tie analysis."""
import heapq
from itertools import combinations

VERTICES = list("ABCDEFGH")
DISTINCT = [("A", "B", 4), ("A", "C", 3), ("B", "C", 5), ("B", "D", 7), ("C", "D", 6), ("C", "E", 8), ("D", "E", 2),
            ("D", "F", 9), ("E", "F", 10), ("E", "G", 11), ("F", "G", 1), ("F", "H", 12), ("G", "H", 13), ("B", "H", 14)]
TIED = [(u, v, {("B", "C"): 4, ("E", "F"): 9}.get((u, v), w)) for u, v, w in DISTINCT]   # creates equal-weight alternatives



class DisjointSet:
    def __init__(self, items):
        self.parent = {x: x for x in items}

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x
```

```
    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        self.parent[rb] = ra
        return True


def kruskal(edges, reverse_ties=False, trace=False):
    key = (lambda e: (e[2], e[0], e[1])) if not reverse_ties else (lambda e: (e[2], -ord(e[0]), -
ord(e[1])))
    sets, chosen = DisjointSet(VERTICES), []
    for u, v, w in sorted(edges, key=key):
        accepted = sets.union(u, v)
        if trace:
            print(f"  consider {u}-{v} ({w}): {'ACCEPT' if accepted else 'reject (cycle)'}")
        if accepted:
            chosen.append((u, v, w))
    return chosen


def prim(edges, start="A", trace=False):
    graph = {v: [] for v in VERTICES}
    for u, v, w in edges:
        graph[u].append((w, u, v))
        graph[v].append((w, v, u))
    in_tree, chosen, heap = {start}, [], list(graph[start])
    heapq.heapify(heap)
    if trace:
        print(f"  start at {start}")
    while heap and len(in_tree) < len(VERTICES):
        w, u, v = heapq.heappop(heap)
        if v in in_tree:
            continue
        in_tree.add(v)
        chosen.append((u, v, w))
        if trace:
            print(f"  cheapest crossing edge {u}-{v} ({w}) -> tree = {sorted(in_tree)}")
        for item in graph[v]:
            if item[2] not in in_tree:
                heapq.heappush(heap, item)
    return chosen
```

```
def boruvka(edges, prefer_late=False, trace=False):
    """Ties are broken by a fixed total order on edges so that no cycle can form."""
    order = (lambda e: (e[2], e[0], e[1])) if not prefer_late else (lambda e: (e[2], -ord(e[0]), -
ord(e[1])))
    sets, chosen, rounds = DisjointSet(VERTICES), [], 0
    while len({sets.find(v) for v in VERTICES}) > 1:
        rounds += 1
        cheapest = {}
        for e in edges:
            ru, rv = sets.find(e[0]), sets.find(e[1])
            if ru == rv:
                continue
            for r in (ru, rv):
                if r not in cheapest or order(e) < order(cheapest[r]):
                    cheapest[r] = e
        if trace:
            print(f"  round {rounds}: " + "; ".join(f"comp {r}: {e[0]}-{e[1]}({e[2]})" for r, e in
sorted(cheapest.items())))
        for u, v, w in set(cheapest.values()):
            if sets.union(u, v):
                chosen.append((u, v, w))
    return chosen


def describe(edges):
    names = sorted(f"{min(u, v)}{max(u, v)}" for u, v, _ in edges)
    return f"{names} total = {sum(w for _, _, w in edges)}"


def all_minimum_spanning_trees(edges):
    best, found = float("inf"), []
    for subset in combinations(edges, len(VERTICES) - 1):
        sets = DisjointSet(VERTICES)
        if all(sets.union(u, v) for u, v, _ in subset):
            cost = sum(w for _, _, w in subset)
            if cost < best:
                best, found = cost, []
            if cost == best:
                found.append(subset)
    return best, found


if __name__ == "__main__":
    print("Network (distinct weights):", DISTINCT)
```

```
    print("\nKruskal trace:")
    k = kruskal(DISTINCT, trace=True)
    print("\nPrim trace:")
    p = prim(DISTINCT, trace=True)
    print("\nBoruvka trace:")
    b = boruvka(DISTINCT, trace=True)
    for name, res in (("Kruskal", k), ("Prim", p), ("Boruvka", b)):
        print(f"{name:8s}: {describe(res)}")

    print("\nModified network (B-C 5->4, E-F 10->9):", TIED)
    for name, res in (("Kruskal (low-label ties first)", kruskal(TIED)),
                      ("Kruskal (high-label ties first)", kruskal(TIED, reverse_ties=True)),
                      ("Prim from A", prim(TIED, "A")), ("Prim from H", prim(TIED, "H")),
                      ("Boruvka (low-label ties)", boruvka(TIED)), ("Boruvka (high-label ties)",
boruvka(TIED, True))):
        print(f"{name:32s}: {describe(res)}")
    cost, trees = all_minimum_spanning_trees(TIED)
    print(f"Exhaustive check over C(14,7) = 3432 subsets: {len(trees)} distinct MSTs of cost
{cost}")
    for t in trees:
        print("   ", sorted(f"{min(u, v)}{max(u, v)}" for u, v, _ in t))
```

Output 11 – edge-selection traces and final trees

```
Network (distinct weights): [('A', 'B', 4), ('A', 'C', 3), ('B', 'C', 5), ('B', 'D', 7), ('C', 'D', 6), ('C', 'E',
8), ('D', 'E', 2), ('D', 'F', 9), ('E', 'F', 10), ('E', 'G', 11), ('F', 'G', 1), ('F', 'H', 12), ('G', 'H', 13), ('B',
'H', 14)]

Kruskal trace:
  consider F-G (1): ACCEPT
  consider D-E (2): ACCEPT
  consider A-C (3): ACCEPT
  consider A-B (4): ACCEPT
  consider B-C (5): reject (cycle)
  consider C-D (6): ACCEPT
  consider B-D (7): reject (cycle)
  consider C-E (8): reject (cycle)
  consider D-F (9): ACCEPT
  consider E-F (10): reject (cycle)
  consider E-G (11): reject (cycle)
  consider F-H (12): ACCEPT
  consider G-H (13): reject (cycle)
  consider B-H (14): reject (cycle)

Prim trace:
```

```
  start at A
  cheapest crossing edge A-C (3) -> tree = ['A', 'C']
  cheapest crossing edge A-B (4) -> tree = ['A', 'B', 'C']
  cheapest crossing edge C-D (6) -> tree = ['A', 'B', 'C', 'D']
  cheapest crossing edge D-E (2) -> tree = ['A', 'B', 'C', 'D', 'E']
  cheapest crossing edge D-F (9) -> tree = ['A', 'B', 'C', 'D', 'E', 'F']
  cheapest crossing edge F-G (1) -> tree = ['A', 'B', 'C', 'D', 'E', 'F', 'G']
  cheapest crossing edge F-H (12) -> tree = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']

Boruvka trace:
  round 1: comp A: A-C(3); comp B: A-B(4); comp C: A-C(3); comp D: D-E(2); comp E: D-
E(2); comp F: F-G(1); comp G: F-G(1); comp H: F-H(12)
  round 2: comp A: C-D(6); comp D: C-D(6); comp F: D-F(9)
Kruskal : ['AB', 'AC', 'CD', 'DE', 'DF', 'FG', 'FH'] total = 37
Prim    : ['AB', 'AC', 'CD', 'DE', 'DF', 'FG', 'FH'] total = 37
Boruvka : ['AB', 'AC', 'CD', 'DE', 'DF', 'FG', 'FH'] total = 37
```

Algorithm	**Next safe edge**	**Information maintained**	**How the partial solution grows**
Kruskal	The globally cheapest remaining edge that joins two different trees (cycle test).	Edges sorted by weight; a disjoint-set structure of the current components.	A forest: separate trees appear (F–G, D–E, A–C) and merge as edges are accepted (C–D, D–F).
Prim	The cheapest edge leaving the single current tree (the cut between the tree and the rest).	A set of vertices in the tree and a priority queue of frontier edges.	One tree from A: A–C, A–B, C–D, D–E, D–F, F–G, F–H.
Borůvka	For every component, its own cheapest outgoing edge, all chosen in the same round.	A component label for every vertex and the cheapest edge found per component.	Round 1 adds A–C, A–B, D–E, F–G, F–H (8 components fall to 3); round 2 adds C–D and D–F.

Final result. All three algorithms return the same MST: A–B, A–C, C–D, D–E, D–F, F–G and F–H, with **total cost 37**. The traces differ substantially: Kruskal examines 14 edges and rejects 7 as cycle-forming, such as B–C (5), which would close the triangle A, B, C; Prim grows one tree and needs seven selections; Borůvka finishes in only two rounds. This shows that equal final costs hide very different construction behaviour.

Stress test – multiple MSTs with the same cost

Two weights were changed to create ties: B–C from 5 to 4 (it now ties with A–B at 4) and E–F from 10 to 9 (it now ties with D–F at 9). The edge A–C (3) joins A and C, so either A–B or B–C connects B; and D–E (2) joins D and E, so either D–F or E–F connects the pair to F and G. The

tie-break rule of each implementation was varied (low or high vertex labels first, different Prim start vertices).

Output 12 – tie experiments and exhaustive check

```
Modified network (B-C 5->4, E-F 10->9): [('A', 'B', 4), ('A', 'C', 3), ('B', 'C', 4), ('B', 'D', 7), ('C', 'D', 6), ('C', 'E', 8), ('D', 'E', 2), ('D', 'F', 9), ('E', 'F', 9), ('E', 'G', 11), ('F', 'G', 1), ('F', 'H', 12), ('G', 'H', 13), ('B', 'H', 14)]
Kruskal (low-label ties first)  : ['AB', 'AC', 'CD', 'DE', 'DF', 'FG', 'FH'] total = 37
Kruskal (high-label ties first) : ['AC', 'BC', 'CD', 'DE', 'EF', 'FG', 'FH'] total = 37
Prim from A                     : ['AB', 'AC', 'CD', 'DE', 'DF', 'FG', 'FH'] total = 37
Prim from H                     : ['AB', 'AC', 'CD', 'DE', 'DF', 'FG', 'FH'] total = 37
Boruvka (low-label ties)        : ['AB', 'AC', 'CD', 'DE', 'DF', 'FG', 'FH'] total = 37
Boruvka (high-label ties)       : ['AC', 'BC', 'CD', 'DE', 'EF', 'FG', 'FH'] total = 37
Exhaustive check over C(14,7) = 3432 subsets: 4 distinct MSTs of cost 37
   ['AB', 'AC', 'CD', 'DE', 'DF', 'FG', 'FH']
   ['AB', 'AC', 'CD', 'DE', 'EF', 'FG', 'FH']
   ['AC', 'BC', 'CD', 'DE', 'DF', 'FG', 'FH']
   ['AC', 'BC', 'CD', 'DE', 'EF', 'FG', 'FH']
```

Can the algorithms return different edge sets and all still be correct? Yes. Kruskal with high-label ties first returned {A–C, B–C, C–D, D–E, E–F, F–G, F–H}, whereas with low-label ties it returned {A–B, A–C, C–D, D–E, D–F, F–G, F–H}; Borůvka reproduced the same two sets depending on its tie-break; Prim returned the first set from both start vertices tested. The cost is 37 in every case. An exhaustive check of all C(14,7) = 3432 seven-edge subsets confirms that exactly **four** distinct spanning trees have the minimum cost of 37, namely the four combinations of {A–B or B–C} with {D–F or E–F}, and every output of every algorithm is one of them. The explanation is the cut and cycle property: an edge is safe if it is a minimum-weight edge across some cut, and when two edges across a cut have equal weight, either one is safe, so the algorithm's tie-break rather than the graph decides which tree is returned. The MST is unique only when all weights are distinct, as in the first graph. A practical warning for Borůvka: with equal weights and no consistent tie-break, each of three components of a triangle could choose a different tied edge in the same round and create a cycle, which is why the implementation uses a fixed total order.

Industry decision – three deployment environments

- **A. Sparse network (E close to V).** Kruskal is the natural choice. Sorting E edges is cheap when E is small, the union-find structure is nearly constant time per operation, and the edge-list representation needs no adjacency structure. Prim with a binary heap is also efficient, and Kruskal has the further advantage that edges can be streamed in sorted order and the work can stop as soon as V−1 edges are accepted (my implementation examines all 14 edges for the sake of a complete trace, but a production version can stop after V−1 acceptances, which on large sparse inputs leaves a long tail of edges unexamined).
- **B. Dense network (E close to V²).** Prim is preferable. It never needs to sort all of the edges: it only maintains, for each outside vertex, its cheapest connection to the tree, so with an adjacency matrix it runs in $O(V^2)$ and does not pay the cost of sorting about V^2 edges.

Kruskal's sort and the long run of rejected cycle-forming edges (7 of 14 in my small trace, a fraction that rises as the graph gets denser) become the bottleneck.

- **C. Large network on parallel or distributed resources.** Borůvka is the right design. In each round every component's cheapest outgoing edge can be found independently, so the edge list can be partitioned across machines and only the component labels need to be exchanged; the number of components at least halves each round (8 → 3 → 1 in my trace), giving about log V rounds. Kruskal depends on a single global sorted order and a shared union-find state that every acceptance changes, so it is inherently sequential, and Prim grows one tree through a shared frontier. Distributed Borůvka does need a deterministic tie-break order on edges across all machines; otherwise the cycle problem described above appears.

Task 6 – Last-Mile Delivery Route Optimisation: TSP by Dynamic Programming (9 Marks)

6.1 Modelling the Optimisation Problem

- **Input:** a distribution hub (location 0), n delivery locations numbered 1 to n, and a matrix cost[a][b] giving the known travel cost from a to b (it need not be symmetric).
- **Output:** a route (an ordering of the n locations) that starts and ends at the hub, together with its total cost.
- **Objective:** minimise the sum of costs of the consecutive legs hub → p1 → p2 → … → pn → hub.
- **Constraints:** the route starts at the hub, visits every delivery location exactly once, and returns to the hub; costs are non-negative and known in advance.

Why the current location alone is not enough. Two partial routes can end at the same location and still be different problems. Hub → A → B and Hub → C → B both end at B, but the first still has to visit C and the second still has to visit A. The set of locations that remain to be visited changes which completions are legal, so the state must include the set of visited locations as well as the current location. The order in which the visited locations were seen does not matter for the future, only their cost so far, so among all partial routes with the same visited set and the same current location, only the cheapest needs to be kept. That is the key idea that allows DP to replace the n! permutations.

6.2 Design of the DP State

- **DP[S][j]** stores the minimum cost of a path that starts at the hub, visits exactly the locations in S (each once) and ends at location j ∈ S.
- **Base state:** for a single location, S = {j}: DP[{j}][j] = cost[hub][j].
- **Building larger states:** DP[S][j] = min over k ∈ S\{j} of (DP[S\{j}][k] + cost[k][j]). The route to j is the best route to some other location k covering all of S except j, plus the last leg k → j.
- **Final return:** the optimal tour cost is min over j of (DP[full][j] + cost[j][hub]), where full is the set of all n locations; the return leg is added only once, at the end.

The subset S is stored as a bit mask, so that S\{j} is S with bit j cleared (mask XOR bit). A parent table records the best k for every state, and the route is reconstructed by starting from the best final j, stepping to parent[full][j], and removing j from the mask each time. The recurrence has

$n \cdot 2^{(n-1)}$ states and each state tries up to $n-1$ predecessors, so the time is $O(n^2 \cdot 2^n)$ and the memory is $O(n \cdot 2^n)$.

6.3 Implementation of the Route Optimisation Engine

Listing 6 – task6_tsp.py (source code)

```
"""Task 6 - Last-mile delivery route by Held-Karp dynamic programming over subsets."""
import itertools
import random
import time
import tracemalloc

INF = float("inf")



def solve_tsp(cost):
    """cost[0] is the hub; deliveries are 1..n. dp[mask][j]: cheapest hub->...->j visiting exactly the set 'mask'."""
    n = len(cost) - 1
    if n < 1:
        raise ValueError("need at least one delivery location")
    full = (1 << n) - 1
    dp = [[INF] * n for _ in range(1 << n)]
    parent = [[-1] * n for _ in range(1 << n)]
    states = transitions = 0
    for j in range(n):                          # base states: hub -> j directly
        dp[1 << j][j] = cost[0][j + 1]
        states += 1
    for mask in range(1, 1 << n):
        for j in range(n):
            if not mask & (1 << j) or mask == (1 << j):
                continue
            states += 1
            previous = mask ^ (1 << j)
            for k in range(n):
                if previous & (1 << k):
                    transitions += 1
                    candidate = dp[previous][k] + cost[k + 1][j + 1]
                    if candidate < dp[mask][j]:
                        dp[mask][j], parent[mask][j] = candidate, k
    best, last = min((dp[full][j] + cost[j + 1][0], j) for j in range(n))   # close the tour at the hub
    route, mask = [], full
    while last != -1:
        route.append(last + 1)
        last, mask = parent[mask][last], mask ^ (1 << last)
```

```
    return best, [0] + route[::-1] + [0], {"states": states, "transitions": transitions, "table_cells": (1
<< n) * n}


def brute_force(cost):
    n = len(cost) - 1
    results = []
    for order in itertools.permutations(range(1, n + 1)):
        path = (0,) + order + (0,)
        results.append((sum(cost[a][b] for a, b in zip(path, path[1:])), path))
    return sorted(results)


def random_costs(n, seed):
    rng = random.Random(seed)
    m = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        for j in range(n + 1):
            if i != j:
                m[i][j] = rng.randint(5, 40)
    return m


def label(i):
    return "Hub" if i == 0 else chr(ord("A") + i - 1)


def show_matrix(cost):
    print("      " + " ".join(f"{label(j):>4}" for j in range(len(cost))))
    for i, row in enumerate(cost):
        print(f"{label(i):>5} " + " ".join(f"{c:>4}" for c in row))


if __name__ == "__main__":
    cost6 = random_costs(6, seed=2026)
    print("6.3 Cost matrix (hub + 6 locations):")
    show_matrix(cost6)
    best, route, stats = solve_tsp(cost6)
    print("Route:", " -> ".join(label(i) for i in route))
    print(f"Minimum Travel Cost = {best}; stats = {stats}")

    print("\n6.4 Validation, hub + 4 locations")
    cost4 = random_costs(4, seed=99)
    show_matrix(cost4)
```

```
    ranked = brute_force(cost4)
    print(f"{len(ranked)} tours enumerated; cheapest five:")
    for c, path in ranked[:5]:
        print(f"   cost {c}: " + " -> ".join(label(i) for i in path))
    print(f"Worst tour cost: {ranked[-1][0]}")
    dp_cost, dp_route, _ = solve_tsp(cost4)
    print(f"DP: cost {dp_cost}, route " + " -> ".join(label(i) for i in dp_route), "| equals brute
force:", dp_cost == ranked[0][0])
    agree = sum(solve_tsp(random_costs(n, s))[0] == brute_force(random_costs(n, s))[0][0] for n
in range(2, 8) for s in range(30))
    print(f"Agreement on random instances n=2..7: {agree} of {6 * 30}")

    print("\n6.5 State-space growth")
    print(f"{'n':>3} {'states':>9} {'transitions':>12} {'table cells':>12} {'time (s)':>9} {'peak
MB':>8} {'n!':>16}")
    for n in range(4, 17):
        cost = random_costs(n, seed=n)
        tracemalloc.start()
        t0 = time.perf_counter()
        _, _, st = solve_tsp(cost)
        elapsed = time.perf_counter() - t0
        peak = tracemalloc.get_traced_memory()[1] / 1e6
        tracemalloc.stop()
        factorial = 1
        for k in range(2, n + 1):
            factorial *= k
        print(f"{n:>3} {st['states']:>9} {st['transitions']:>12} {st['table_cells']:>12}
{elapsed:>9.3f} {peak:>8.1f} {factorial:>16}")
    print("Projected states for n=200: n*2^(n-1) = about", f"{200 * 2 ** 199:.2e}")
```

Dataset: one hub and six delivery locations A to F with the asymmetric random cost matrix below (generated with seed 2026, costs between 5 and 40).

Output 13 – cost matrix and optimal route

```
6.3 Cost matrix (hub + 6 locations):
      Hub    A    B    C    D    E    F
 Hub    0   12   25   37   37   11   19
   A   40    0   31   40   36   33   20
   B    5   10    0   12   23   11   33
   C    5   36   25    0   18   30   21
   D   27   27   29   37    0    9   26
   E   10   40   39   23   23    0   34
   F   14   24    6   28   28   34    0
Route: Hub -> A -> F -> B -> C -> D -> E -> Hub
Minimum Travel Cost = 87; stats = {'states': 192, 'transitions': 480, 'table_cells': 384}
```

Result: Hub → A → F → B → C → D → E → Hub with **minimum travel cost 87**. Checking by hand against the matrix: 12 (Hub→A) + 20 (A→F) + 6 (F→B) + 12 (B→C) + 18 (C→D) + 9 (D→E) + 10 (E→Hub) = 87. The order of visits is A, F, B, C, D, E, and the return to the hub is included.

6.4 Validating the Solution

For a smaller instance (hub and four locations) the 4! = 24 possible tours were enumerated exhaustively by brute force and sorted by cost. The best five and the worst tour are shown, followed by the DP result.

Output 14 – exhaustive validation

```
6.4 Validation, hub + 4 locations
      Hub   A   B   C   D
 Hub    0  30  29  17  16
   A   19   0  20  13  10
   B   21  29   0  38  39
   C   10  36  17   0  31
   D   18  28  29  18   0
24 tours enumerated; cheapest five:
  cost 91: Hub -> C -> B -> A -> D -> Hub
  cost 95: Hub -> D -> A -> C -> B -> Hub
  cost 96: Hub -> A -> D -> C -> B -> Hub
  cost 96: Hub -> B -> A -> D -> C -> Hub
  cost 97: Hub -> D -> B -> A -> C -> Hub
Worst tour cost: 145
DP: cost 91, route Hub -> C -> B -> A -> D -> Hub | equals brute force: True
Agreement on random instances n=2..7: 180 of 180
```

Comparison. The independent reference result is a minimum of 91, reached by the tour Hub → C → B → A → D → Hub (17 + 17 + 29 + 10 + 18 = 91). The DP produced exactly the same cost and the same route. The DP also agreed with brute force on all 180 random instances tested (30 instances for each of n = 2 to 7). **How this increases confidence:** the brute-force enumerator is a different algorithm that shares no logic with the DP; it is simple enough to be obviously correct; agreement on the cost and on the route in the case shown, and agreement on 180 other instances, makes it very unlikely that the base states, the mask arithmetic, the recurrence or the final return leg are wrong. A defect such as forgetting the return leg or mishandling the parent pointers would have produced a mismatch with a tour cost somewhere among the 24, and the spread (91 to 145) shows the check is not trivial. This does not prove correctness for larger n, but the bit-mask logic is identical for all n.

6.5 Investigating State-Space Growth

The program counts the number of DP states evaluated, the number of transitions (the inner-loop evaluations), the table cells allocated, the running time (with memory tracing active, so times are slightly inflated) and the peak memory, for n = 4 to 16.

Output 15 – state-space growth table

```
6.5 State-space growth
 n    states  transitions  table cells  time (s)  peak MB            n!
 4        32           48           64     0.000      0.0            24
 5        80          160          160     0.001      0.0           120
 6       192          480          384     0.002      0.0           720
 7       448         1344          896     0.005      0.0          5040
 8      1024         3584         2048     0.014      0.1         40320
 9      2304         9216         4608     0.029      0.1        362880
10      5120        23040        10240     0.095      0.3       3628800
11     11264        56320        22528     0.267      0.6      39916800
12     24576       135168        49152     0.670      1.3     479001600
13     53248       319488       106496     0.974      2.8    6227020800
14    114688       745472       229376     3.189      5.8   87178291200
15    245760      1720320       491520     6.136     12.1  1307674368000
16    524288      3932160      1048576    15.874     25.2 20922789888000
Projected states for n=200: n*2^(n-1) = about 1.61e+62
```

Observations. The state count matches $n \cdot 2^{(n-1)}$ exactly (for n = 4: 32; for n = 16: 524,288). Each additional location roughly doubles the work: from n = 15 to n = 16 the states grew by a factor of 2.13, the transitions by 2.29, the time from 6.1 s to 15.9 s (a factor of 2.6) and the memory from 12.1 MB to 25.2 MB (a factor of 2.1). The doubling is the signature of the number of subsets, 2^n.

Inquiry – why DP removes repeated computation but is still expensive. Brute force examines n! complete tours ($16! \approx 2.1 \times 10^{13}$ for n = 16). Many of these share the same partial route; for instance all orderings of the remaining locations after the same prefix repeat the same sub-calculation. DP computes the best route for each (subset, last location) pair once and reuses it, so for n = 16 it evaluated 524,288 states, a reduction of about eight orders of magnitude. But the number of different subproblems is not small: every subset of the n locations can occur as a visited set, and there are 2^n subsets. DP replaces a factorial by an exponential, and an exponential is still far beyond reach for large n. The table must also hold a cell for each subset and end location, so memory grows with the same 2^n factor and is the first resource to be exhausted.

6.6 Business Requirement Change – 200 Locations

- **Why exact DP becomes impractical.** Using the state formula validated in 6.5, n = 200 gives about $200 \times 2^{199} \approx 1.6 \times 10^{62}$ states. Since my own measurements show about 16 s and 25 MB at n = 16 and a doubling for each extra location, the exact method would need astronomical time and memory long before reaching n = 200. No machine can store that table or complete the computation within a few seconds.
- **Property of the requirement that changed:** the requirement has moved from “provably minimum cost” to “a very good route within a few seconds”. The optimality guarantee has been relaxed and a time limit has been imposed, so the problem is now one of bounded-time approximation rather than exact optimisation.

- **Strategy to investigate:** heuristic and approximation methods. A constructive heuristic such as nearest neighbour or greedy edge insertion produces a route in $O(n^2)$, and a local-search improvement such as 2-opt or Or-opt, or the more powerful Lin–Kernighan style moves, refines it repeatedly until the time budget expires. Metaheuristics such as simulated annealing or genetic algorithms are further candidates. If the costs satisfy the triangle inequality, an approximation algorithm such as Christofides gives a proven bound.
- **Trade-off the company accepts:** it gives up the guarantee of optimality, since the route may be a few per cent more expensive than the best possible, and it cannot know exactly how far from optimal the result is unless a bound is available. In return it obtains a result in seconds, a quality that can be improved by allowing more time, and a method that scales to hundreds or thousands of locations.

6.7 Technical Reflection (not more than 150 words)

Dynamic programming beats enumeration for TSP because it stores the best cost for each (subset, end-location) pair instead of recomputing every permutation. My implementation shows this: for n = 16 it evaluated 524,288 states, whereas enumeration needs $16! \approx 2.1 \times 10^{13}$ tours. However, this does not make DP scalable in general. The number of states is n·2^(n−1), and each added location roughly doubled my states, running time (6.1 s to 15.9 s from n = 15 to 16) and memory (12.1 MB to 25.2 MB). The table needs a cell for every subset, so memory becomes the limit. DP is efficient only when the number of distinct subproblems is polynomial, as in the knapsack table of n × W cells or the 2n states of assembly-line scheduling. TSP's subproblem space is exponential, so DP merely turns a factorial into an exponential.

Debugging Log and Defects Found during Development

The assignment stresses that a program that runs is not finished. The following problems were actually found while building and testing the six programs, with the way each was diagnosed and corrected. They are included because the DEBUG and DEFEND stages of the workflow ask for evidence of how correctness was established, not only the final outputs.

Task	Defect or weak result found	Diagnosis and correction
1	First experiment at W = 24 showed greedy first-fit and DP giving the same penalty (93), so it did not demonstrate the value of DP.	A sweep over many widths located W = 33 and W = 34, where greedy gives 89 and 118 against 65 and 94 for DP; the sweep was added to the program so the evidence is reproducible.
2	The "greedy or best single item" repair left the number of suboptimal instances unchanged (662 of 2000).	Reported honestly in 2.4 instead of being presented as an improvement; the recommendation was left as plain ratio greedy.
3	A common variant of Dijkstra that re-pushes a vertex whenever its distance improves would silently repair the negative-edge case in some graphs and hide the failure.	The implementation uses an explicit finalised set, which is the true assumption of the algorithm, so the failure of Stage B is visible and can be compared with Bellman-Ford. The negative edge was chosen with its head

		finalised before its tail and no path back, so no cycle is created.
4	My first choice for Task 4.3 raised a transfer time that the optimal route does not use (the index of a transfer list is “leave line l after station i”), so the route and the total stayed at 38 and proved nothing.	The trace table showed which transfers the base route used; the experiment was repeated with the Line 2 to Line 1 transfer after station 2, which is on the optimal route, and the route then changed as predicted.
5	With equal weights Borůvka can select a different tied edge in each component and form a cycle.	A fixed total order (weight, then vertex labels) is applied in every round; the exhaustive count of 4 MSTs then confirmed that every output was a valid minimum tree.
6	Possible defects in mask arithmetic, base states or the final return leg of the TSP DP.	Checked against brute force on the 4-location instance and on 180 random instances (n = 2 to 7); all matched, and the state counts agree exactly with n·2^(n−1).

Evidence Checklist against the Mandatory Submission Chain

The assignment requires the chain Source Code → Test Input → Expected Behaviour → Actual Output → Intermediate Algorithmic Evidence → Boundary/Failure Test → Interpretation. The table shows where each link is found for every task.

Task	Source code and input	Intermediate evidence	Boundary or failure test	Prediction or change test	Interpretation
1	Listing 1; 14-word text	DP table with options per state (Output 1)	Word = width, empty, long word (Output 2)	W = 20, 28, 36 table (1.3)	1.3 and 1.4 text
2	Listing 2; P1–P4 and A–D	Decision sequences; counters (Outputs 3, 4)	Counterexample 42 against 48	Fast-answer requirement (2.4)	2.3 table and 2.4
3	Listing 3; 8 vertices, 13 edges	Finalisation order, predecessors, matrices	Negative edge and negative cycle	Stage B prediction	Stage B to D text
4	Listing 4; 6 stations per line	Stay and transfer trace (Output 9)	Unequal-line input rejected	4.3 prediction and 4.4 disruption	4.2 to 4.5 text
5	Listing 5; 8 nodes, 14 edges	Edge-selection traces (Output 11)	Ties and 4 MSTs (Output 12)	Modified weights	Industry decision

6	Listing 6; hub and 6 locations	State and transition counters (Output 15)	Brute force, 180 instances	200-location requirement (6.6)	6.5 to 6.7 text

Overall Conclusion and Reproducibility

Across the six tasks the same engineering lesson appears. A program that runs is not evidence of correctness: the evidence came from boundary cases (a word exactly as wide as the line, an invalid station count, empty text), from adversarial inputs designed to break the method (the knapsack counterexample and the negative edge that silently fools Dijkstra), from independent validation (brute force against DP for TSP, Bellman-Ford against Floyd), and from instrumentation that exposed what each algorithm was doing (DP tables, traces, operation counts and state counts). The requirement-change questions in Tasks 1.3, 2.4, 4.3, 4.4 and 6.6 were answered from measured behaviour: width changes did not always lower the penalty, greedy was fast but only about 97.6% of optimal on average, a changed transfer time moved the route by one decision, and the TSP state space doubled with each new location.

Reproducibility. Each listing is a single file, run with the command “python3 taskN_name.py”. All random inputs use fixed seeds (Task 2: 7 and 11; Task 6: 2026, 99 and n), so the outputs shown in this report can be regenerated exactly, apart from the running times in Task 6.5, which depend on the machine.

Stated assumptions and limits. Task 1 assumes single spaces and rejects words longer than the width. Task 2 assumes integer weights for the DP table. Task 3 uses directed edges and treats an unreachable vertex as infinity. Task 4 allows one transfer between consecutive stations. Task 5 assumes a connected graph. Task 6 allows asymmetric costs and was measured only up to 16 locations, because the exact method doubles in cost with every extra location. Running times were measured in Python and show relative growth rather than the speed of an optimised implementation.